

一、个人信息

姓名	郑建辉	邮箱	xechoz.zheng@gmail.com
技术方向	Android 客户端开发、核心业务开发、业务架构设计、跨模块通信、状态管理、AI Coding	手机	18825111003
学历	本科 · 计算机科学与技术	毕业院校	华南师范大学（2011.09 - 2015.06）

二、个人简介

10年以上移动端开发经验，过去8年主要负责 HAGO 及多款出海社交产品的登录、首页、语聊、社区等核心模块，参与需求分析、技术方案设计和上线交付，支撑产品在东南亚、中东、南美等市场持续迭代。在业务开发过程中，也负责模块化、跨模块通信、状态管理和双端网络层等架构建设，相关方案覆盖100+业务模块。近两年参与团队AI Coding实践，将现有架构规范整理为可复用的AI Skill，保障生成代码的一致性。

三、专业技能

- **业务领域：** 海外社交 / 休闲游戏 App、语聊房、直播、登录链路、商业化（支付 / 广告）、多市场合规
- **客户端技术：** Android（Kotlin / Java）、iOS（Objective-C）、协程、Jetpack Compose、MVVM、单向数据流、模块化 / 组件化、跨模块通信、状态管理
- **基础建设：** Protobuf 代码生成、双端网络层、KSP 编译期框架、路由组件、SDK 统一接入（Google / Facebook / TikTok、Google Play IAP、Pangle / AdMob / Meta）
- **AI Coding：** 团队 AI Skill 体系、AI-Driven 架构约束、Copilot / Cursor / OpenCode 工作流

四、工作经历

4.1 欢聚时代（JOYY） / 高级工程师 / 2018 - 2026.6

HAGO——海外休闲游戏社交 App，覆盖语聊房、直播、休闲小游戏、社交、商业化等业务形态，月活千万级，面向东南亚、中东、南美等多市场；工作范围同时覆盖 infun、ola、bolohi 等 App。

- **核心业务开发：** 登录、首页、语聊、社区四大核心模块的功能开发与迭代；首页沉淀为统一框架，被 infun、ola、bolohi 复用
- **基础架构与模块化（2018-2023）：** MVVM 架构、App 主页与主模块架构；模块化与跨模块通信的服务发现与管理核心实现
- **架构与基建（2023-2026）：** 双端一致性网络库（Objective-C/Java）、iOS 路由组件、单向数据流状态管理框架、团队 AI Coding 基建

4.2 早期经历（2015-2018）

- 荔枝 FM / Android 开发工程师（2017-2018）：音频/直播内容平台，负责直播间架构设计与业务开发
- 唯品会 / Android 开发工程师（2015-2017）：电商 App 业务开发

五、项目经验

5.1 登录（多 App 多市场统一登录链路，2018-2026）

业务背景 HAGO 及 infun、ola 等产品覆盖东南亚、中东、南美市场，各市场主流登录渠道与合规要求不同，产品形态差异导致登录方式组合各异；出海场景弱网、域名被墙常见，登录链路不稳定直接抬高用户进入成本。

方案 多 App、多市场登录方式统一与配置化，部分市场因合规要求需禁用特定渠道，渠道按市场裁剪；渠道 SDK 模块独立编译，按 App 与市场按需引入；登录容错链路——SDK 异常降级与兼容、弱网自动重试、域名不可达时切换备用域名/IP 兜底；多次失败后引导切换其他登录方式，仍失败则降级游客登录，保障用户可进入产品、避免流失。

成果 新市场/新 App 接入渠道配置即生效；容错链路支撑千万月活的多市场登录，海外弱网场景登录稳定性得到保障。

5.2 语聊房业务开发：麦位模块（2019-2025）

业务背景 语聊房是 HAGO 核心业务形态，麦位是房间内最核心的实时交互：上下麦、抢麦、连麦等操作在弱网下既要响应快，又要保证多端状态一致。

方案 麦位状态同步收敛为**后台为准**：前期客户端为快速响应采用假成功（本地先变更、后台确认），导致多端状态不一致，改为服务端确认后更新；针对频繁刷新的性能问题，改为增量刷新、合并刷新，高频请求加缓存，减少无效流量与卡顿；UI 复用上，为加快房间打开曾整体复用房间 UI，但多数业务不支持重入与幂等，收敛为业务按需复用——麦位等模块实现完整复用。

成果 麦位状态不一致问题消除，房间高频操作场景性能与稳定性提升。

5.3 多 App 首页与多形态扩展（2019-2026，持续演进）

业务背景 首页是千万月活 App 的首屏：语聊房、直播、游戏、社区，运营位和布局的调整频繁。各个 App 形态不一样——有主做游戏分发的，有做社交的，有走内容流的。早期首页缺少代码上的规划，导致需求要写多遍，新 App 立项首页需从零开始。也考虑过全部配置化，但推荐流的形态差异迭代太快，配置表达不了，配置维护成本反而比直接写更高。

方案 在容器与推荐流之间做了边界的划分：App 容器层统一，布局、运营位、业务入口走配置；推荐流层不配置化，框架只约定数据和视图的接入方式，各 App 按自己的形态写自己的流。

成果 从 2019 年持续使用并演进到现在：后续立项的 infun、ola、bolohi 等新项目，首页无需单独排期，配好容器、接上自家推荐流即可用；运营位和布局改动一次配置、各 App 生效。各 App 的推荐流各自演化，不受框架迭代牵连。

5.4 跨模块通信与 MVVM 框架（2019-2025，持续演进）

业务背景 业务从单体扩张至 100+ 模块，各业务线并行迭代、能力互用需求增加；此前模块间通信靠人肉约定、无编译期保障，联调频繁出错，且模块内结构各异、协作成本高。

方案 负责整体架构概要设计与方案对齐，主导模块化架构设计：基于 KSP 编译期生成服务注册与发现代码，接口与实现编译期校验，避免运行时反射开销；模块内统一 MVVM，ViewModel 绑定业务 Context 而非页面生命周期，提供一致的子功能组织与数据共享方式。

成果 团队 20+ 开发者日常使用，新增模块只需定义接口与实现；通信由人肉约定转为编译期保障，语聊、直播、商业化等业务线可并行迭代、能力互用，新人接手成本降低。框架持续演进至 2025 年，承载 HAGO 全部 100+ 业务模块，并支撑 infun、ola、bolohi 等 App 快速立项上线。

5.5 业务状态管理容器（单向数据流，2024-2026）

业务背景 直播间、主播主页、多级导航等高频刷新场景状态流复杂、刷新粒度粗、生命周期靠手动管理，线上问题反复、排查成本高；各模块状态管理各写各的。

方案 设计细粒度状态订阅：状态按属性分发而非整对象通知，减少冗余刷新；协程生命周期与状态管理器自动绑定，免手动管理、降低泄漏风险；与既有 MVVM、Service 体系原生集成，跨页面/模块共享同一套 setState 规范。

成果 统一 Compose 与传统 View 的状态消费 API，一套 ViewModel 在两种 UI 下行为一致；已覆盖直播间、主页、导航等核心模块，此类线上问题减少，高频场景稳定性与可维护性提升。

5.6 Android & iOS 双端业务网络层（跨端方案统一，2023-2024）

业务背景 Android 与 iOS 各自维护独立网络层，接口靠文档传递，一次协议变更两端对齐联调耗时数天，是跨端协作主要瓶颈。

方案 以 Protobuf 协议文件为唯一来源，编写代码生成工具自动产出 Android/iOS 两端网络调用代码，保证双端处理一致，业务方统一接口调用、无需关注平台差异。

成果 覆盖双端登录、语聊、社区等全部核心模块。协议变更只需更新 proto 重新生成，双端联调从数天缩短至小时级，消除接口不一致导致的生产事故。

5.7 商业化 SDK 统一接入体系（2018-2026）

业务背景 面向海外多市场，需接入 Google/Facebook/TikTok 登录、Google Play 内购及 Pangle/AdMob/Meta 广告等 SDK，各家 API 形态各异、迭代频繁，业务直接依赖导致耦合高、升级与隐私合规改造成本大。

方案 设计统一 SDK 接入层屏蔽各家差异，业务侧通过统一接口接入：**登录** Google/Facebook/TikTok 归一为统一登录态；**支付** Google Play 内购统一订单与发货校验；**广告** Pangle/AdMob/Meta 统一抽象，激励视频/插屏/Banner 回调归一，支持兜底降级；SDK 服务异常时可通过配置禁用对应业务入口。

成果 业务接入成本降低，登录与广告渠道可替换、可并行接入；第三方模块编译期按需引入、运行期按线上配置启停，适配各地区合规与业务需求。

5.8 团队 AI Coding 基建：基础架构&规范 建设（2024-2026）

业务背景 团队广泛使用 Copilot、Cursor、OpenCode 等 AI 编码工具，但生成代码缺乏架构约束、频繁偏离 MVVM、Service Manager、单向数据流等规范，Review 大量耗时在架构对齐。

方案 负责将 HAGO 核心架构规范转化为结构化 AI Skill，约束 AI 生成代码遵循团队架构；推动「需求 AI 出计划 → 基于 Skill 生成代码 → Bug AI 辅助定位」工作流落地。并以 Vibe Coding 验证边界：基于 OpenCode + 大模型 API，2 周内完成 4 款小游戏 Demo 全流程开发，印证架构约束的必要性；验证也发现部分架构约束 AI 无法稳定遵循，收敛为人工关键点 Review。

成果 20 人团队日常使用，AI 生成代码架构符合率提升，Review 返工率降低，整体交付效率提升约 30%。